Table 1: Qualitative examples where SWE-PBT-CORE generates stronger tests than EOTTER.

Instance ID / Aspect	eotter	swe-pbt-core

`django__django-12143`
Input diversity
	Uses one hardcoded prefix copied from the issue:
	`prefix = 'test(prefix).with + special*chars'`
	Line 66. Only this exact combination is tested.
		Generates 200 examples using all 13 regex special characters:
		`special_char = data.draw(`
		`st.sampled_from(regex_special_chars))`
		The special character is injected at a random position in a randomly generated base prefix (*Lines 133–145*).

`django__django-12143`
Code under test
	The test constructs the escaped pattern itself:
	`pk_pattern = re.compile(`
	`r'{}-\d+-{}$'.format(`
	`re.escape(prefix), pk_name))`
	Line 90. The test therefore applies the intended `re.escape()` fix itself rather than exercising Django's production implementation.
		Calls Django's actual production method:
		`result =`
		`admin._get_edited_object_pks(`
		`request, prefix_with_special)`
		Line 157. This directly exercises the method containing the bug.

`django__django-12143`
Assertion quality
	The escaped pattern is checked with:
	`self.assertIsNotNone(`
	`pk_pattern.search(form_field_name))`
	The buggy result is computed:
	`match_result =`
	`buggy_pattern.search(form_field_name)`
	but never asserted. The test also contains:
	`self.assertTrue(True)`
	(*Lines 94, 102, 106*).
		Checks Django's real output directly:
		`assert pk_value in result`
		(*Line 161*).
		On buggy code, the regex mismatch produces an empty result and the assertion fails; on fixed code, it passes.

`sympy__sympy-13372`
Input diversity
	Tests two hardcoded orderings:
	`Mul(x, Max(0,y),`
	`evaluate=False).evalf()`
	and
	`Mul(Max(0,y), x,`
	`evaluate=False).evalf()`
	(*Lines 20–25*).
	Only the specific `Max(0,y)` configuration from the issue is explored.
		Randomizes the number of symbols:
		`num_symbols = data.draw(`
		`st.integers(min_value=1, max_value=3))`
		and samples four variants:
		`Max(0,y), Max(y,0),`
		`Min(0,y), Min(y,0)`
		The generated function is placed before the symbol:
		`expr = Mul(unevaluatable_func,`
		`x, evaluate=False)`
		(*Lines 45–46, 84–92*).

Table 2: Qualitative examples where EOTTER generates stronger tests than SWE-PBT-CORE.

Instance ID / Aspect	**eotter**	**swe-pbt-core**

`django__django-15851`
Oracle strength
	Checks the complete `(args, env)` tuple:
	`self.assertEqual(`
	`self.settings_to_cmd_args_env(...),`
	`(["psql", ..., "-c", ..., "dbname"],`
	`{"PGPASSWORD": "somepassword"}))`
	Checks ordering, required arguments, and environment together.
		Only checks relative parameter position:
		`if dbname in args:`
		`for param in parameters:`
		`if param in args:`
		`assert args.index(param) <`
		`args.index(dbname)`
		Missing `dbname` or parameters can skip the assertion.

`django__django-15851`
Environment validation
	Explicitly validates:
	`{"PGPASSWORD": "somepassword"}`
	Both key and value are checked.
		Environment is returned but not validated:
		`args, env =`
		`DatabaseClient.settings_to_cmd _args_env(...)`
		No assertion checks `env`.

`django__django-15554`
Bug context
	Uses the project's actual models:
	`from .models import Author, Book, Editor`
	and two filtered aliases:
	`book_editor_a = FilteredRelation(...)`
	`book_editor_b = FilteredRelation(...)`
	Matches the schema of the reported bug.
		Uses custom models:
		`class Zone(models.Model): ...`
		`class Product(models.Model): ...`
		`class Price(models.Model): ...`
		Tests a different schema from the reported bug.

`django__django-15554`
Query verification
	Checks the JOIN structure:
	`join_count = sql.lower().count(`
	`"left outer join")`
	`self.assertGreaterEqual(join_count, 2)`
	and executes the query:
	`result = qs.first()`
	`self.assertIsNotNone(result)`
	SQL structure and runtime behavior are both tested.
		Checks only the generated SQL:
		`join_count = sql_query.upper().count(`
		`"LEFT OUTER JOIN")`
		`assert join_count >= 2`
		The query is never executed and annotation values are not checked.